EXAMEN 1: OPTIMIZACIÓN Y MÉTODOS COMPUTACIONALES

ESTUDIANTE: HÉCTOR MANUEL SALAZAR RIVERA **CÓDIGO:** 1004735481

Complejidad Temporal BUBBLE SORT

Mejor caso (Un Arreglo Ordenado)

Este hará la comparación de cada uno de los números hasta llegar a n, por lo tanto, su complejidad es:

$$T(n) = n - 1$$
$$T(n) = O(n)$$

Peor caso (Arreglo ordenado Inversamente)

- **Bucle interno (j):** Pasadas a través de la lista, se ejecuta $n - 1$ veces.
- **Bucle externo (i):** Comparaciones entre elementos adyacentes en el bucle interno i.

$$T(n) = n - 1 - i$$

El número total de comparaciones es la suma de las iteraciones del bucle interno para cada pasada del bucle externo.

$$T(n) = \sum_{i=0}^{n-2}(n - 1 - i)$$

Expandiendo la sumatoria de $i = 0$ hasta $n - 1$:

$$T(n) = \sum_{i=0}^{n-2}(n - 1 - i)$$

$$T(n) = (n - 1) + (n - 2) + (n - 3) + \cdots + 2 + 1$$

Si aplicamos la fórmula de suma aritmética Gaussiana:

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

El término predominante en esta última ecuación es n^2. Por lo tanto:

$$T(n) = O(n^2)$$

Complejidad Espacial BUBBLE SORT

El algoritmo se ejecuta y va cambiando de orden los datos guardados en la memoria original, por lo que su complejidad es de orden constante, $S(n) = O(1)$.

Análisis de Correctitud mediante LOOP INVARIANT

Para analizar este ciclo tomaremos un arreglo $A[n-i \dots n-1]$ que se encuentra completamente ordenado y contiene los i elementos de mayor magnitud.

```
1: BubbleSort(A)
2:   n = A.length
3:   for i = 0 to n - 1
4:     for j = 0 to n - i - 2
5:       if A[j] > A[j + 1]
6:         swap(A[j], A[j + 1])
```

- **Se define el tamaño del problema (n).**
- **Inicialización:** Antes de la primera iteración ($i = 0$), el arreglo A ordenado es un conjunto vacío, por lo que está ordenado por definición.
- **Finalización:** El ciclo concluye cuando $i = n$; el subarreglo A está ordenado, lo que garantiza que este arreglo solucionó el problema.
- **Mantenimiento (Inicio):** El ciclo interno recorre la porción no arreglada del arreglo.
- **Mantenimiento (Comparación):** Evalúa los elementos adyacentes del arreglo para comparar su magnitud.
- **Mantenimiento (Cierre):** El intercambio mueve el valor desordenado hacia la posición $n - i - 1$. Al concluir el ciclo de la línea 4, el tamaño del subarreglo ordenado aumenta a $i + 1$, pasando a la siguiente comparación.

Complejidad Temporal QUICK SORT

El Quicksort en su forma recursiva de ”divide y vencerás” consta de un proceso iterativo que recorre el arreglo linealmente, este con un costo de $\Theta(n)$.

La ecuación de Quicksort es:

$$T(n) = T(k) + T(n - k - 1) + \Theta(n)$$

Donde k es el tamaño del subarreglo.

Peor caso (Arreglo ordenado Inversamente y se selecciona el último elemento como pivote)

Ecuación del peor caso:

$$T(n) = T(n - 1) + T(0) + cn$$

Donde:

- $T(n-1)$: Tiempo en ordenar de forma recursiva el subarreglo que contiene todos los elementos menos el pivote.
- $T(0)$: Tiempo para procesar la partición vacía (constante).
- cn: Tiempo para recorrer el arreglo de tamaño n comparando cada elemento con el pivote.

Al eliminar la constante obtenemos la ecuación:

$$T(n) = T(n-1) + cn$$

Donde, al sustituir $T(n-1)$ por su definición ($T(n-2) + c(n-1)$), obtenemos:

$$T(n) = [T(n-2) + c(n-1)] + cn$$

Y si continuamos esta expansión hasta el caso base $T(1)$, al ser una progresión aritmética usamos la sumatoria y factorizamos el costo c:

$$T(n) = T(1) + c(2) + c(3) + \cdots + c(n-1) + cn$$

$$T(n) = c\sum_{i=1}^{n} i$$

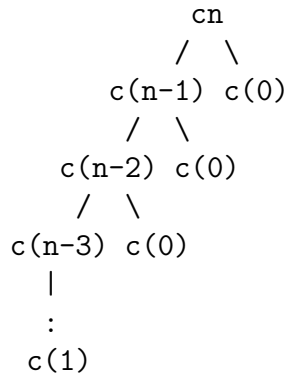
Al aplicar la fórmula de Gauss de una sumatoria obtenemos:

$$T(n) = \frac{c}{2}n^2 + \frac{c}{2}n$$

Al descartar los coeficientes y los elementos de crecimiento de menor orden, obtenemos que la complejidad temporal es de orden:

$$T(n) \in \Theta(n^2)$$

Esto lo podemos ver en el siguiente árbol de secuencias:



Esto se debe a que, al elegir un pivote máximo o mínimo, el subarreglo de la izquierda siempre va a estar vacío, obligando a todos los n a pasar por un solo subarreglo.

Mejor caso (Un Arreglo Ordenado pivote en $n/2$)

Ecuación para el mejor caso:

$$T(n) = 2T(n/2) + cn$$

Donde:

- $2T(n/2)$: El tiempo para ordenar de forma recursiva los dos subarreglos que tienen la mitad del tamaño del arreglo original ($n/2$).
- cn: Costo de recorrer el arreglo de tamaño n comparando los elementos con el pivote para organizarlos a la derecha o a la izquierda.

En la ecuación de recurrencia evaluamos $T(n/2)$:

$$T(n/2) = 2T(n/4) + c(n/2)$$

Reemplazando esto en la ecuación original obtenemos:

$$\begin{aligned} T(n) &= 2[2T(n/4) + c(n/2)] + cn \\ T(n) &= 4T(n/4) + cn + cn \\ T(n) &= 4T(n/4) + 2cn \end{aligned}$$

Al evaluar $T(n/4)$ en la ecuación de recurrencia:

$$T(n/4) = 2T(n/8) + c(n/4)$$

Y reemplazando este valor en la ecuación anterior:

$$\begin{aligned} T(n) &= 4[2T(n/8) + c(n/4)] + 2cn \\ T(n) &= 8T(n/8) + cn + 2cn \\ T(n) &= 8T(n/8) + 3cn \end{aligned}$$

Se puede notar un patrón que aumenta en cada nivel del arreglo; a este nivel lo llamamos k. Por lo tanto, si reemplazamos k en la ecuación original obtenemos:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn$$

Esta ecuación se detiene cuando llega al elemento base $T(1) = 1$, reemplazamos y obtenemos:

$$\frac{n}{2^k} = 1 \implies n = 2^k \implies k = \log_2(n)$$

Y ahora despejamos k en la ecuación original:

$$T(n) = 2^{\log_2(n)} T(1) + (\log_2(n))cn$$

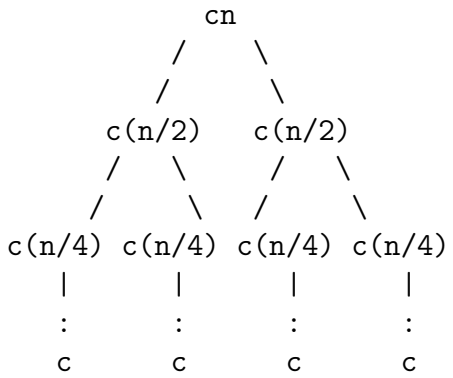
Por propiedad logarítmica, donde una base elevada al logaritmo de la misma base se anula dejando solo el argumento:

$$T(n) = n \cdot c_0 + cn \log_2(n)$$

Al descartar las constantes obtenemos que la complejidad temporal es logarítmica:

$$T(n) = \Theta(n \log n)$$

Esto lo podemos observar en el siguiente árbol de secuencias:



En este caso, el pivote está en la mitad, por lo que el arreglo se divide en partes iguales. Por lo que en cada nivel se hace un tiempo cn, este se divide a la mitad en cada paso hasta llegar a un tamaño 1, por lo que su altura se expresa como $\log_2(n)$.

Complejidad Espacial QUICK SORT

En este algoritmo el uso de la memoria depende del caso al cual se presente:

- **Peor caso:** Cuando el pivote es el máximo o mínimo de un arreglo ordenado inversamente. En este caso, en vez de dividir el problema por la mitad solo nos vamos por una ramificación, esto lo hace de manera lineal, donde se crea una llamada de la función recursiva por cada paso del algoritmo. Esta es del mismo tamaño de los elementos; por lo tanto, es de orden $\Theta(n)$.
- **Mejor caso:** Cuando el algoritmo escoge un buen pivote. Al hacer esto, el problema se divide en dos mitades exactas. Esto genera que el espacio usado sea logarítmico, ya que en cada ramificación este guarda su resultado y se llama en la ramificación adyacente (eso se ve de mejor manera en el árbol de secuencias). Por lo tanto, su complejidad espacial es de orden $\Theta(\log n)$.